



TÉCNICO SUPERIOR EN DESARROLLO DE
APLICACIONES MULTIPLATAFORMA
Departamento de Informática

PROYECTO



Manual Técnico

Autor: Antonio Carretero Guerrero
Curso Académico: 2024/2025

Índice

1 Introducción	3
2 Objetivos	4
3 Tecnologías involucradas	5
3.1Tecnologías aprendidas en el ciclo	5
3.2Tecnologías utilizadas para el proyecto	7
4 Proceso de desarrollo	9
4.1 Análisis	9
4.1.1 Diagrama de casos de uso	10
4.1.2 Diagrama Entidad Relación	10
4.1.3 Diagrama relacional	11
4.2 Desarrollo	11
4.2.1 Diagrama de secuencias	13
4.2.1 Diagrama de secuencias	13
4.2.2 Estructura del proyecto	14
4.3 Pruebas realizadas	18
5 Proceso de Despliegue	21
6 Propuesta de mejora o trabajos futuros	23
7 Bibliografía	24

1 Introducción

Este proyecto surge a partir de la experiencia personal durante el Grado Superior en Laboratorio de Análisis y Control de Calidad (LACC), donde fui testigo directo de las dificultades que suponía el control del almacén químico en el entorno académico. La gestión de los reactivos se realizaba de forma manual, en papel o mediante hojas de cálculo poco centralizadas, lo que con frecuencia ocasionaba pérdidas de inventario, errores en el registro y dificultades para localizar sustancias específicas.

Esta situación, común en muchos laboratorios educativos o pequeños centros de investigación, motivó el desarrollo de Quimisan: una herramienta digital pensada para facilitar y modernizar la gestión del almacén químico, reduciendo errores humanos, centralizando la información y mejorando la trazabilidad de los productos químicos.

En el siguiente documento se detalla la información técnica relativa al proyecto desarrollado, con el objetivo de facilitar su comprensión, mantenimiento y posible evolución futura.

Para el trabajo final del módulo de Aplicaciones Multiplataforma, “**Quimisan**”, se ha llevado a cabo la implementación de una aplicación de escritorio multiplataforma, basada en tecnologías como **React**, **TypeScript**, **Electron**, **Node.js** y **SQLite**. La aplicación está destinada a la **gestión de un almacén químico**, proporcionando una interfaz funcional, clara y segura para la administración de reactivos y usuarios.

Este sistema ha sido diseñado para su uso en entornos de laboratorio o almacenes donde se requiera un control riguroso del inventario químico. La aplicación permite registrar y consultar reactivos, gestionar su entrada y salida, controlar su estado, fechas de caducidad, proveedor y otros metadatos relevantes. Además, incluye un sistema de control de usuarios con distintos roles, así como un historial de acciones que proporciona trazabilidad sobre los movimientos y modificaciones realizadas.

El sistema está orientado a garantizar una experiencia organizada y eficiente tanto para los administradores del almacén como para el personal autorizado, optimizando la gestión interna y mejorando la seguridad y el control sobre las sustancias almacenadas.

2 Objetivos

La finalidad principal de la aplicación desarrollada para este proyecto es la de facilitar y optimizar la gestión del almacén químico, proporcionando un entorno centralizado, seguro y eficiente para el control de reactivos, movimientos e información relacionada.

Se pretende que esta aplicación sirva principalmente para conseguir:

- **Registro detallado de reactivos**, incluyendo nombre, número CAS, cantidad, unidad, proveedor, tipo, fecha de caducidad, y ficha de seguridad (FDS).
- **Control de entradas y salidas de sustancias**, registrando cada movimiento con su cantidad, usuario responsable, fecha y descripción.
- **Trazabilidad total del inventario**, permitiendo conocer el estado actual y pasado de cada reactivo gracias a un sistema de historial.
- **Gestión de usuarios con diferentes roles** (administrador y usuario general), asegurando permisos diferenciados según el nivel de responsabilidad.
- **Prevención de pérdidas o extravíos de material**, evitando errores derivados de una gestión manual o desorganizada.
- **Centralización y digitalización de la información**, reemplazando métodos tradicionales como hojas de cálculo o registros en papel.
- **Mejora de la seguridad y organización** en el entorno de laboratorio, al tener siempre acceso a datos actualizados y fiables sobre las sustancias almacenadas.

Con todo ello, el objetivo final es lograr una herramienta que aporte **control**, **orden y eficiencia** al trabajo diario en laboratorios que manejan productos químicos.

3 Tecnologías involucradas

3.1 Tecnologías aprendidas en el ciclo

- Lenguajes de programación y de marcado

• Java	
• Python	
• XML	
• JavaFx	
• HTML y CSS	
• Json	

Proyecto “Desarrollo de Aplicaciones Multiplataforma”

Título del Proyecto: Quimisan



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

- **Lenguajes para base de datos relaciones y no relacionales**

• SQL	
• SQLite	
• MySQL	
• MongoDB	

- **Entornos de desarrollo(IDE)**

• IntelliJ	
• Pycharm	
• Visual Studio Code	
• MySQL workbench	
• MongoDB Compass	
• Android Studio	

3.2 Tecnologías utilizadas para el proyecto

Tras un análisis inicial de los requerimientos de la aplicación y en base a la experiencia adquirida durante el ciclo, así como a las preferencias personales de desarrollo, se ha decidido implementar la aplicación “**Quimisan**” con el uso de las siguientes tecnologías y herramientas:

- **Entorno de desarrollo:**

Se ha utilizado el editor **Visual Studio Code**, por su ligereza, versatilidad y compatibilidad con múltiples tecnologías web y de escritorio, así como su integración con sistemas de control de versiones como Git.

- **Lenguajes de programación:**

El proyecto se ha desarrollado utilizando:

- **TypeScript** para la parte del frontend en React, lo cual aporta mayor robustez, tipado estático y mejor mantenibilidad del código.
- **JavaScript** para el backend desarrollado con Node.js, incluyendo la gestión de la lógica de Electron.
- **HTML y CSS** para la estructura y estilos básicos de la interfaz, junto con librerías adicionales de diseño.

- **Frameworks y tecnologías principales:**

- **React** (con archivos .tsx) para el desarrollo de la interfaz de usuario.
- **Electron** para empaquetar la aplicación como una aplicación de escritorio multiplataforma.
- **Node.js** para la ejecución del backend, gestión de archivos y conexión con la base de datos.
- **SQLite** como base de datos local y ligera, integrada directamente en el sistema sin necesidad de servidores externos.

- **Base de datos:**

La información del sistema se almacena en una base de datos **SQLite**, ideal por su sencillez, rapidez y capacidad de integrarse fácilmente con aplicaciones de escritorio. Se han creado diversas tablas para gestionar reactivos, movimientos, usuarios, historial y recuperación de contraseñas.

- **Librerías adicionales:**

- **Nodemailer:** utilizada para el envío de correos electrónicos en el proceso de recuperación de contraseña.
- **bcrypt:** empleada para el cifrado seguro de contraseñas de los usuarios.
- **vanta.js:** librería utilizada para aplicar efectos visuales animados al fondo de la interfaz de usuario, mejorando así la experiencia visual de la aplicación.

- **Control de versiones:**

Durante todo el desarrollo se ha utilizado **GitHub** como plataforma para el control de versiones, permitiendo un respaldo continuo del proyecto, seguimiento de cambios y organización del código fuente.

4 Proceso de desarrollo

4.1. Análisis

4.1.1. Diagrama de casos de uso

En este diagrama se quieren resumir las posibilidades que tiene un usuario que utiliza *Quimisan* .

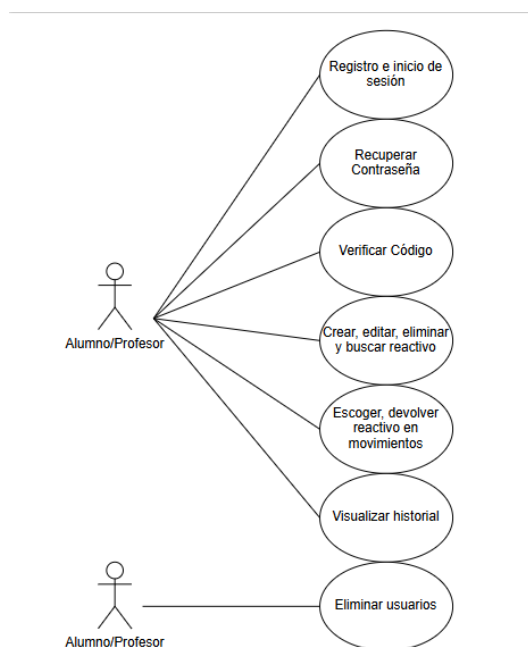


Figura 1: Diagrama de casos de uso

4.1.2. Diagrama de flujo

En este diagrama se representa el ciclo de vida de *Quimisan*

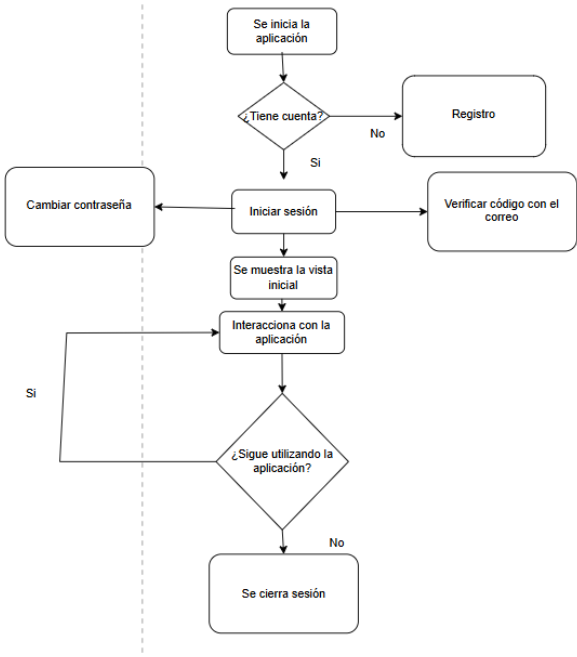


Figura 2: Diagrama de flujo

4.1.2. Diagrama Entidad-Relación

Sinterización simple del diagrama de clases de una base de datos.

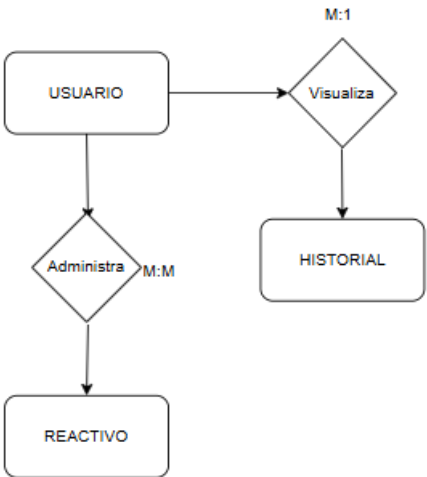


Figura 3: Diagrama de Entidad-Relación

4.1.3. Diagrama relacional

Muestra la forma en la que se relacionan las tablas en la base de datos y los diferentes datos que almacena cada tabla, así como el tipo de dato.

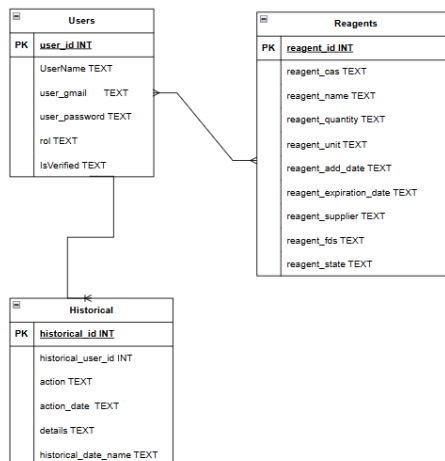


Figura 4: Diagrama relacional

4.2. Desarrollo (diagrama de secuencias,...)

El desarrollo de la aplicación *Quimisan* se ha llevado a cabo siguiendo una arquitectura basada en capas, donde se separan claramente las responsabilidades entre la interfaz de usuario, la lógica de negocio y el acceso a datos.

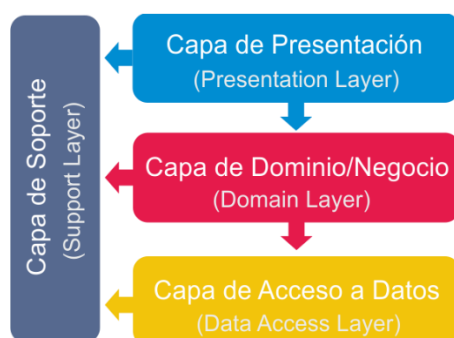


Figura 5: Arquitectura en capas

➤ Arquitectura general

La aplicación se compone de tres capas principales:

- **Capa de presentación** (Frontend): desarrollada con **React** y **TypeScript**, gestionando las vistas y la interacción con el usuario a través de componentes .tsx y hojas de estilo .css.
- **Capa lógica** (Backend local): implementada con **Node.js** y **JavaScript**, encargada de procesar las peticiones del frontend, acceder a la base de datos, aplicar validaciones, manejar usuarios, movimientos, etc.
- **Capa de persistencia**: basada en **SQLite**, una base de datos ligera y local que permite mantener el control completo de los datos del almacén químico sin necesidad de un servidor externo.

Estas capas están integradas mediante **Electron**, que permite empaquetar la aplicación como una solución de escritorio multiplataforma, funcionando de forma autónoma en diferentes sistemas operativos.

➤ Flujo de funcionamiento

Inicialización:

- Al arrancar la aplicación, Electron ejecuta main.js, que a su vez abre la conexión con SQLite y crea las tablas si no existen.
- Se genera la ventana principal y se carga el frontend React a través del archivo HTML especificado.

Comunicación entre capas:

- El frontend se comunica con la lógica del backend mediante el uso de ipcRenderer e ipcMain (canales de Electron).
- Por ejemplo, al registrar un reactivo, el frontend emite un evento (add-reagent), que es interceptado en el backend, donde se realiza la validación y el guardado en SQLite.

Gestión de usuarios y permisos:

- Se implementa un sistema de usuarios con roles (alumno, profesor, administrador), almacenados en la tabla users.
- Las contraseñas se almacenan cifradas mediante la librería **bcrypt**.
- Se controla el acceso a funciones según el rol.
-

Sistema de historial y trazabilidad:

- Cada acción importante (entrada/salida de reactivos, eliminación de usuarios, etc.) se registra en la tabla historial para asegurar trazabilidad y control de auditoría.

Notificaciones por correo:

- Se implementa un sistema de recuperación de contraseña por correo electrónico usando la librería **nodemailer**, con generación de tokens y expiración.

Diseño visual:

- Se ha mejorado la interfaz con la ayuda de la librería **vanta.js** para fondos animados, brindando una experiencia visual moderna.

4.2.1 Diagrama de secuencias

Diagrama de secuencias que representa la estructura de la aplicación

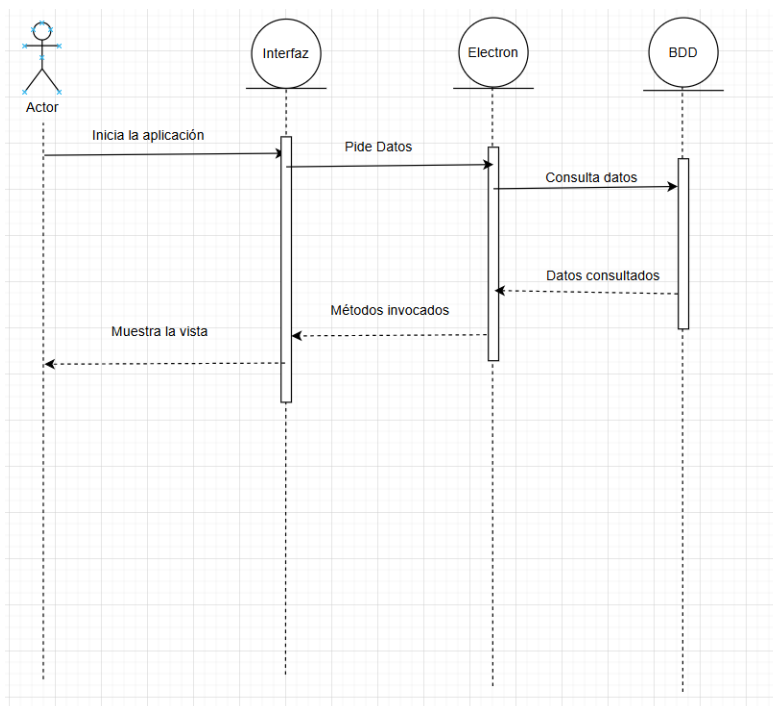


Figura 6: Diagrama de secuencias

4.2.2 Estructura del proyecto

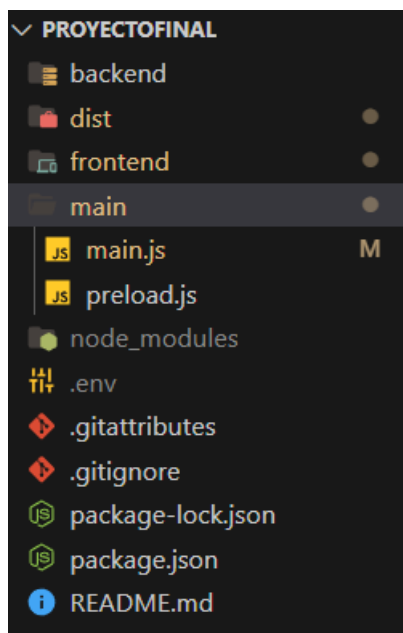


Figura 7: Estructura de carpetas inicial

Quimisan está compuesto inicialmente por cuatro carpetas importantes.

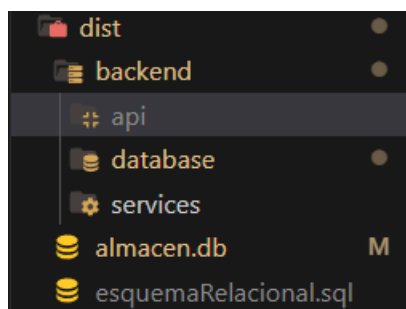


Figura 8: Estructura de carpetas de backend

- **Dist:** La carpeta dist contiene el código transpilado y listo para ejecución en el entorno de producción. En este proyecto, en concreto, la carpeta incluye archivos JavaScript generados a partir del código fuente escrito en TypeScript, en la carpeta backend inicial que se ve en la figura7. Estos archivos representan la lógica del backend y la conexión con la base de datos.

De momento no es necesario explicar la carpeta inicial de backend, ya que esta ni siquiera es utilizable de momento, sin embargo fue mi carpeta inicial con la lógica inicial y acabé utilizando construyendo los servicios en javascript. Recalco “construyendo” ya que fue realizado con un comando.

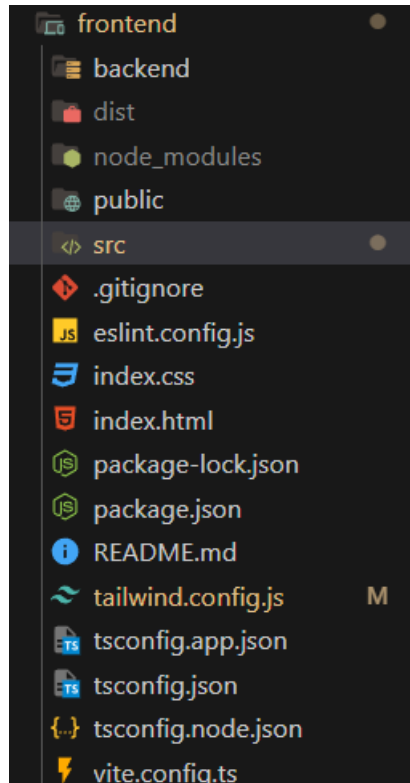


Figura 9: Estructura de carpetas de frontend

- Frontend:

El frontend de la aplicación ha sido desarrollado utilizando React con TypeScript, lo que permite una estructura modular y tipada, facilitando el mantenimiento y escalabilidad del proyecto. A continuación, se describe la organización general de las carpetas y su propósito:

- **Src:**

Carpeta principal del frontend, que contiene todo el código fuente.

Dentro de ella se organiza el proyecto en distintas subcarpetas:

- **Pages**

Carpeta donde se encuentran las diferentes vistas o pantallas principales de la aplicación, como el login, la vista de gestión de reactivos, el panel de administración de usuarios, etc. Cada vista representa una página funcional completa dentro de la interfaz.

- **Styles**

Contiene los archivos de estilos CSS (o SCSS) utilizados para dar apariencia visual a los componentes y vistas. Se puede organizar por archivos globales y específicos por componente o vista.

- **Interfaces**

Contiene las definiciones de interfaces TypeScript utilizadas en el proyecto. Estas interfaces definen las estructuras de los objetos utilizados en la lógica y la comunicación con el backend, como User, Reagent, Movement, etc.

- **Utils**

Funciones auxiliares reutilizables en distintas partes del frontend, en este caso validaciones.

- **Assets**

Recursos estáticos como imágenes, logos o fondos (por ejemplo, si usaste vanta.js para animaciones de fondo, puedes tener archivos de configuración aquí)

- Main:**

Esta carpeta contiene los archivos fundamentales para la inicialización y gestión del proceso principal de Electron. Es donde se configura la ventana de la aplicación, se gestiona la comunicación entre el frontend y el backend, y se establece el entorno seguro de ejecución.

○ **Main.js**

Es el punto de entrada principal de la aplicación Electron. Su función principal es:

- Crear la ventana de la aplicación (BrowserWindow).
- Cargar el frontend (HTML/React).
- Inicializar la base de datos y los servicios del backend.
- Gestionar eventos del ciclo de vida de la aplicación (app.on('ready'), window-all-closed, etc.).
- Registrar los canales de comunicación con el frontend mediante ipcMain.

Es básicamente el "servidor" dentro del entorno de escritorio de Electron.

○ **Preload.js**

- Es un script que se ejecuta antes de que la página web se cargue en la ventana del navegador de Electron. Sus funciones son:
- Actuar como puente seguro entre el proceso principal y el renderizado (frontend).
- Exponer ciertas funciones de Node.js al frontend de forma controlada mediante contextBridge.
- Facilitar la comunicación entre el frontend (React) y el backend (Node.js + base de datos) usando ipcRenderer.

Es esencial para mantener la seguridad, ya que impide que el frontend acceda directamente a Node.js, reduciendo riesgos de inyecciones o ataques.

4.3. Pruebas realizadas

Las pruebas se han realizado de manera manual e incremental, validando cada nueva función a medida que se integraba.

Para optimizar la detección de errores y el análisis, se utilizaron herramientas específicas para cada capa tecnológica y entorno:

Herramientas y su uso

- Consola de Desarrollo (DevTools) en React/Electron:
Para monitorear errores, warnings y trazas en tiempo real durante la ejecución de la interfaz.
- Se usó para revisar el estado y props de los componentes React, especialmente la manipulación de los objetos usuarios, reactivos e historial.
- Validar que la interfaz renderice correctamente los datos recibidos desde el backend.
- Logs en main.js y preload.js (Electron):
- Se insertaron console.log para rastrear el flujo de ejecución de la aplicación Electron, especialmente para la comunicación entre procesos (main y renderer).
- Verificación del correcto paso de datos entre backend y frontend a través de la API de Electron (IPC).
- Depuración de posibles problemas en la inicialización y carga de la app de escritorio.

Consola de Node.js para backend:

- Se revisaron los logs generados por el servidor Node.js para confirmar el procesamiento correcto de las peticiones.
- Se detectaron posibles errores de conexión a la base de datos o fallos en las consultas asociadas a usuarios, reactivos e historial.

Monitoreo de peticiones con herramientas de red en Electron:

- Se utilizó la pestaña de "Network" en DevTools de Electron para verificar que las llamadas fetch o IPC funcionaran correctamente y que los tiempos de respuesta fueran adecuados.

Ejemplos de pruebas realizadas

Usuarios:

- Crear, editar y eliminar usuarios desde el frontend React, verificando el flujo de datos hasta la base de datos.
- Confirmar la actualización inmediata del estado en React y en el historial de acciones.
- Validar que los errores de validación (e.g., usuario sin nombre) se manejen y muestren en la UI.
- Reactivos:
- Obtener listas completas y filtradas mediante consultas backend.
- Actualizar atributos y asignarlos a usuarios, comprobando sincronización con la base de datos.

Historial:

- Registrar eventos generados por acciones de usuarios sobre reactivos.
- Visualizar el historial actualizado en la interfaz y garantizar la persistencia de registros.

Resultados

- Se detectaron problemas de sincronización entre procesos en Electron que fueron corregidos usando la comunicación IPC adecuada.
- Validaciones implementadas en backend Node.js aseguraron datos consistentes y evitaron fallos en la UI React.
- Se mejoró la gestión de errores y mensajes para el usuario final, tanto en frontend como en backend.

5 Proceso de Despliegue

Software necesario

Para poder desplegar y ejecutar la aplicación se requiere:

- **Node.js**
Versión recomendada: 18.x o superior
Descarga oficial: <https://nodejs.org/>
 - **npm** (gestor de paquetes incluido con Node.js)
Versión recomendada: 9.x o superior
 - **Git** (para clonar el repositorio)
Descarga oficial: <https://git-scm.com/>
 - **Electron**
La versión utilizada está especificada en el archivo package.json y se instalará automáticamente durante la instalación de dependencias.
 - **Editor de código (opcional, para desarrollo)**
Recomiendo Visual Studio Code: <https://code.visualstudio.com/>
-

Proceso detallado para el despliegue

Clonar el repositorio

Para obtener el código fuente, clona el repositorio ejecutando en la consola:

Por ejemplo:

```
git clone https://tu-repositorio-url.git
cd nombre-del-proyecto
```

Instalar dependencias del backend y frontend

La aplicación está estructurada para instalar las dependencias tanto del frontend (React + TypeScript), como del backend (Node.js) y Electron.

Para ello, ejecuta:

```
npm install
```

React

Para compilar el frontend y generar una versión optimizada para producción, ejecuta:

```
npm run build
```

Ejecutar la aplicación en modo desarrollo

Para correr la aplicación localmente y hacer pruebas, ejecuta:

```
npm start
```

Construir la aplicación Electron para distribución

Para empaquetar la aplicación Electron y generar un instalador o ejecutable para el sistema operativo, ejecuta:

```
npm run electron-pack
```

Ejecutar la aplicación empaquetada

El ejecutable generado se encontrará en la carpeta dist o similar, dependiendo de la configuración utilizada. Simplemente ejecuta el archivo para iniciar la aplicación de escritorio.

6 Propuesta de mejora o trabajos futuros

Exportación e importación de datos

- Añadir opciones para exportar datos (usuarios, reactivos, historial) en otros formatos como PDF.
- Permitir la importación masiva de datos para facilitar la actualización y carga inicial.

Dashboard con métricas y estadísticas

- Crear una sección visual donde se muestren gráficos y estadísticas relevantes, como número de usuarios activos, uso de reactivos, tendencias en el historial, etc.

Soporte multi-idioma (internacionalización)

- Implementar la capacidad de cambiar el idioma de la interfaz para llegar a un público más amplio. Usar librerías como i18next.

Integración con otros servicios

- Facilitar la integración con servicios externos relevantes, como sistemas de notificación por correo, APIs de terceros para enriquecimiento de datos, o plataformas de análisis.

Mejoras en la gestión de usuarios y reactivos

- Añadir perfiles de usuario más completos con fotos, detalles personales, y preferencias.
- Permitir etiquetas o categorías para reactivos para mejor organización y búsqueda.

7 Bibliografía

- Documentación oficial de Electron: <https://www.electronjs.org/>
- Documentación oficial de React: <https://es.react.dev/>
- Página de w3school con React: <https://www.w3schools.com/REACT/DEFAULT.ASP>
- Página de w3shool con Javascript: <https://www.w3schools.com/js/>
- Página para los fondos de pantalla: <https://www.vantajs.com/>
- Github: <https://github.com/>
- Db Browser para sqlite: <https://sqlitebrowser.org/>
- Documentación para la resolución de problemas: <https://stackoverflow.com/questions>